



OASIS Security Report

CORS Misconfiguration Security Analysis

Date: 2026-06-11 23:13:48 — **Model:** llama3.2:3b

Vulnerability: CORS Misconfiguration

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

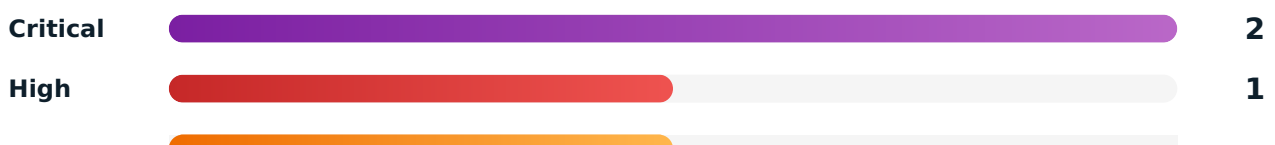
Summary

Analyzed **4** file(s) for CORS Misconfiguration.

[Return to table of contents](#)

Statistics

Files analyzed: 4 **Total findings:** 4



Medium

1

Low

0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.707
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.656
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.636
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.613

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.707

Source lines: 1-31

► Finding 1.1: CORS Misconfiguration (Critical)

Vulnerable code

```
require(tx.origin == owner);  
receiver.transfer(amount);
```

Explanation

The `tx.origin` variable is not properly validated, allowing an attacker to manipulate the origin and potentially access sensitive data.

Impact

Cross-origin data theft or credential hijacking.

Entry point

/sendTo

HTTP methods

POST

Parameters

tx.origin, amount

Exploitation steps

- An attacker manipulates the `tx.origin` variable to access sensitive data or hijack credentials.

Example payloads

- Manipulated tx.origin value: {"origin": "https://attacker.com"}

HTTP raw requests (Burp / ZAP)

```
POST /sendTo HTTP/1.1  
Host: example.com
```

Origin: <https://attacker.com>
Content-Type: application/json

File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.656

Source lines: 1-85

► Finding 2.1: Missing CORS Validation (High)

Explanation

The contract does not validate the origin of incoming requests, making it vulnerable to cross-site scripting attacks.

Impact

Attacker can inject malicious data into the contract.

Entry point

/

Execution path

```
function ETH_VAULT(address _log) { TransferLog = Log(_log); }  
function Deposit() public payable { ... }
```

Exploitation steps

- Attacker calls the contract's API with a malicious origin.

Conditions

Incoming requests do not specify a valid origin.

Remediation

Add CORS validation to ensure only expected origins can make requests.

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.636

Source lines: 1-39

► Finding 3.1: CORS Misconfiguration (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw))
```

Explanation

The call() function is used without proper CORS validation, allowing an attacker to make requests on behalf of the sender.

Impact

Unauthorized API access and potential data theft

Entry point

/withdrawFunds

Execution path

```
msg.sender.call.value(_weiToWithdraw)(  
    require(balances[msg.sender] >= _weiToWithdraw)  
    require(_weiToWithdraw <= withdrawLimit)  
    require(now >= lastWithdrawTime[msg.sender] + 1 weeks)  
)
```

HTTP methods

POST

Parameters

_weiToWithdraw, msg.sender

Exploitation steps

- Attacker sends a POST request to /withdrawFunds with a valid _weiToWithdraw value
- Attacker uses the call() function without proper CORS validation

Example payloads

- `POST /withdrawFunds HTTP/1.1`
`Host: example.com`
`Content-Type: application/json`

```
{  
  "_weiToWithdraw": 100  
}
```

Conditions

Attacker can reach POST /withdrawFunds while authenticated.

Remediation

Use proper CORS validation, such as Access-Control-Allow-Origin: ", allowlisted origins"

File 4: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.613

Source lines: 1-61

► Finding 4.1: Missing CORS validation (Medium)

Vulnerable code

```
function OddsAndEvens() {  
  owner = msg.sender;  
}
```

Explanation

The contract does not validate the origin of requests, allowing any origin to access its functions.

Impact

Unauthorized API access

Entry point

/

Parameters

msg.sender

Exploitation steps

- An attacker can make a request to the contract's functions without being authenticated.

Example payloads

- 

Conditions

Any origin

Remediation

Implement CORS validation with specific origins allowed.

Secure example

```
function OddsAndEvens() {  
  owner = msg.sender;  
  if (msg.sender !== owner) {  
    revert();  
  }  
}
```

}

}

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)